



**Project: Service Management System**

Submitted By : **Muhammad Hasnain (3361)**  
**Muhammad Hasnain (3309)**  
**Raheel Khan (3310)**

Class : **BSCS 5B (Morning)**

Subject : **Web Engineering**

Date : **May 19<sup>th</sup>, 2026**

Deployment Link : <https://new-project-omega-six.vercel.app/>

Repo Link : <https://github.com/irjiwefiu/Web-Project>

**National University of Modern Languages**

## Table of Contents

|                                                                    |    |
|--------------------------------------------------------------------|----|
| 1. Title Page .....                                                | 1  |
| 2. Table of Contents .....                                         | 2  |
| 3. Abstract / Executive Summary .....                              | 3  |
| 4. Introduction .....                                              | 4  |
| 5. System Requirements & Analysis .....                            | 6  |
| 6. Technology Stack .....                                          | 9  |
| 7. System Design .....                                             | 12 |
| 8. Implementation .....                                            | 16 |
| 9. Screenshots (UI Walkthrough) .....                              | 22 |
| 10. Testing .....                                                  | 26 |
| 11. Setup Guide & Default Login Credentials .....                  | 28 |
| 11.5 Default Login Credentials (Admin, Customer, Technician) ..... | 31 |
| 12. Conclusion & Future Work .....                                 | 32 |
| 13. References .....                                               | 33 |
| 14. Appendices .....                                               | 34 |

### 3. Abstract / Executive Summary

The Service Management Platform is a full-stack web application designed to bridge the gap between customers seeking professional services and skilled technicians offering their expertise. The platform addresses the inefficiencies of traditional service booking methods by providing a centralized digital marketplace that streamlines the entire lifecycle of service requests — from creation and technician discovery to assignment, real-time status tracking, payment processing, and post-service reviews.

Built using React 18 for the frontend and Express.js with TypeORM on the backend, the application leverages a PostgreSQL database to manage users, service categories, service requests, technician profiles, assignments, status histories, and reviews. The system implements three distinct user roles — Admin, Customer, and Technician — each with tailored dashboards and permission sets. Role-based access control (RBAC) is enforced through JWT authentication and middleware-driven authorization.

Key outcomes include a responsive dark-themed UI, a robust RESTful API with 57 endpoints, real-time status tracking with full audit history, dual assignment models (admin-directed and technician-initiated applications), and a demo payment flow. The platform is deployed on Vercel with Neon PostgreSQL, demonstrating cloud-native scalability and serverless architecture readiness.

## 4. Introduction

### 4.1 Background & Motivation

In today's fast-paced world, finding reliable technicians for home repairs, maintenance, and professional services remains a fragmented and frustrating experience. Customers often rely on word-of-mouth referrals, scattered social media posts, or uncertain online listings. Conversely, skilled technicians struggle to find consistent work and manage their schedules effectively. This project was motivated by the need to create a structured, transparent, and efficient platform that connects both parties in a seamless digital environment.

### 4.2 Problem Statement

The absence of a centralized service management system leads to communication gaps, delayed responses, unclear pricing, and lack of accountability. Customers have no way to track service progress or provide structured feedback, while technicians lack tools to manage their availability and workload. Administrators cannot oversee operations or generate insights to improve service quality.

### 4.3 Project Objectives

- Design and implement a role-based web application with three distinct user personas: Admin, Customer, and Technician.
- Provide a secure authentication system using JWT tokens with bcrypt password hashing.
- Enable customers to create, manage, and track service requests with real-time status updates.
- Allow technicians to register, build profiles, manage availability, and apply for service requests.
- Implement a dual assignment model: admin-directed assignment and technician-initiated applications with customer acceptance.
- Provide role-specific dashboards with key metrics and quick actions for each user type.
- Support full CRUD operations for users (admin), categories (admin), and service requests (customer).
- Include a demo payment system and post-service rating/review functionality.
- Deploy the application on a cloud platform using serverless architecture with a managed PostgreSQL database.
- Ensure a responsive, accessible, and visually consistent dark-themed user interface.

### 4.4 Scope of the Project

In Scope:

- User registration and login with role selection (customer/technician)
- Admin management of users, categories, and technicians
- Customer creation, viewing, updating, and cancellation of service requests
- Technician profile management and availability toggling
- Assignment workflows (admin assign, technician apply, customer accept/reject)
- Real-time status tracking with full history audit trail
- Post-service reviews and ratings (1-5 stars)

- Demo payment processing
- Role-specific dashboards with aggregated statistics
- Dark-themed responsive UI

Out of Scope:

- Real-time chat or messaging between customers and technicians
- Live GPS tracking of technicians
- Actual payment gateway integration (demo only)
- Mobile native applications (responsive web only)
- Multi-language support

#### **4.5 Target Audience / End Users**

- Customers: Homeowners, renters, and businesses seeking professional repair or maintenance services.
- Technicians: Skilled professionals (electricians, plumbers, cleaners, mechanics, etc.) offering their services.
- Administrators: Platform operators who manage users, categories, and oversee all operations.

## 5. System Requirements & Analysis

### 5.1 Functional Requirements

| ID    | Feature                | Description                                                                                                        |
|-------|------------------------|--------------------------------------------------------------------------------------------------------------------|
| FR-01 | User Registration      | The system shall allow new users to register as customers or technicians with name, email, password, and username. |
| FR-02 | User Login             | The system shall authenticate users via email and password and issue a JWT token.                                  |
| FR-03 | Role-Based Access      | The system shall restrict access to features based on user role (admin, customer, technician).                     |
| FR-04 | Profile Management     | Users shall be able to view and update their own profile information.                                              |
| FR-05 | Technician Profile     | Technicians shall create and update their bio, skills, service area, and availability status.                      |
| FR-06 | Category Management    | Admins shall create, read, update, and delete service categories.                                                  |
| FR-07 | Create Service Request | Customers shall create service requests with title, description, category, location, urgency, and price.           |
| FR-08 | Manage Requests        | Customers shall view, update, and cancel their own requests.                                                       |
| FR-09 | View All Requests      | Admins shall view and filter all service requests in the system.                                                   |
| FR-10 | Technician Assignment  | Admins shall assign technicians to requests; technicians shall apply for available requests.                       |
| FR-11 | Application Management | Customers shall view technician applications and accept or                                                         |

|              |                    |                                                                                                                                |
|--------------|--------------------|--------------------------------------------------------------------------------------------------------------------------------|
|              |                    | reject them.                                                                                                                   |
| <b>FR-12</b> | Status Tracking    | The system shall track status changes (requested → assigned → on_the_way → in_progress → completed/cancelled) with timestamps. |
| <b>FR-13</b> | Reviews & Ratings  | Customers shall submit 1-5 star ratings with comments for completed services.                                                  |
| <b>FR-14</b> | Payment Processing | Customers shall process demo payments for completed service requests.                                                          |
| <b>FR-15</b> | Dashboard          | Each role shall have a personalized dashboard with relevant statistics and quick actions.                                      |
| <b>FR-16</b> | Search & Filter    | Technicians shall search for available requests by keyword, category, and location.                                            |

## 5.2 Non-Functional Requirements

| ID            | Requirement    | Description                                                                                          |
|---------------|----------------|------------------------------------------------------------------------------------------------------|
| <b>NFR-01</b> | Performance    | API responses shall complete within 500ms for 95% of requests under normal load.                     |
| <b>NFR-02</b> | Security       | Passwords shall be hashed using bcrypt with 10 salt rounds. JWT tokens shall expire within 24 hours. |
| <b>NFR-03</b> | Availability   | The system shall achieve 99.9% uptime through Vercel serverless deployment.                          |
| <b>NFR-04</b> | Scalability    | The serverless architecture shall auto-scale to handle traffic spikes.                               |
| <b>NFR-05</b> | Responsiveness | The frontend shall render                                                                            |

|               |                 |                                                                                                            |
|---------------|-----------------|------------------------------------------------------------------------------------------------------------|
|               |                 | correctly on devices from 320px to 2560px width.                                                           |
| <b>NFR-06</b> | Usability       | The UI shall follow a consistent dark theme with clear navigation and visual feedback.                     |
| <b>NFR-07</b> | Maintainability | The codebase shall use modular architecture with separate controllers, services, repositories, and routes. |
| <b>NFR-08</b> | Data Integrity  | Database constraints and TypeORM entity validation shall prevent inconsistent data states.                 |

### 5.3 User Roles & Permissions

| Role              | Capabilities                                                                      | Dashboard Access     | Management Access                        |
|-------------------|-----------------------------------------------------------------------------------|----------------------|------------------------------------------|
| <b>Customer</b>   | Create/view/cancel requests, accept/reject technicians, submit reviews & payments | Customer Dashboard   | My Requests, Find Technicians, Profile   |
| <b>Technician</b> | Manage profile, toggle availability, apply for requests, update job statuses      | Technician Dashboard | Performance View, Profile                |
| <b>Admin</b>      | Full user CRUD, category CRUD, direct assignment, view all data                   | Admin Dashboard      | Users, Requests, Technicians, Categories |

### 5.4 Hardware & Software Requirements

Hardware Requirements (Development):

- CPU: Dual-core processor (2.0 GHz or higher)
- RAM: Minimum 4 GB (8 GB recommended)
- Storage: 500 MB free disk space
- Display: 1366x768 resolution or higher

Software Requirements:



- Operating System: Windows 10+, macOS 12+, or Linux (Ubuntu 20.04+)
- Node.js: v18.x or higher
- PostgreSQL: v14.x or higher
- npm: v9.x or higher
- Modern Browser: Chrome 90+, Firefox 88+, Edge 90+, Safari 14+
- Git: For version control

### **5.5 Constraints & Assumptions**

- The system assumes users have reliable internet connectivity.
- The payment module is a simulated demo and does not process real transactions.
- The application is optimized for modern browsers; legacy browser support is not guaranteed.
- Database migrations must be run manually during deployment setup.
- The system uses polling (10-second intervals) rather than WebSocket for real-time updates.

## 6. Technology Stack

### 6.1 Frontend Technologies

- **React 18:** A declarative, component-based JavaScript library for building user interfaces. Chosen for its virtual DOM performance, extensive ecosystem, and reusable component model.
- **Vite 5: A** fast build tool and development server with hot module replacement (HMR). Selected over Create React App for significantly faster startup and build times.
- **Redux Toolkit:** State management library that provides a centralized store for application state. Used for managing authentication state and UI state across the application.
- **React Router DOM v6:** Declarative routing library for React. Enables protected routes, nested layouts, and role-based navigation.
- **Tailwind CSS 3:** A utility-first CSS framework that enables rapid UI development with consistent design tokens. Used to implement the custom dark theme.
- **.@tanstack/react-table:** A headless table library for building powerful data tables with sorting, filtering, and pagination. Used for user and request management tables
- **react-icons / tabler-icons / lucide-react:** Icon libraries providing a consistent set of SVG icons for the user interface.
- **date-fns:** A lightweight date utility library for formatting and manipulating dates throughout the application.

### 6.2 Backend Technologies

- **Node.js:** JavaScript runtime built on Chrome's V8 engine. Enables JavaScript to be used on the server, allowing code sharing between frontend and backend.
- **Express.js v5:** A minimal and flexible Node.js web application framework. Provides a robust set of features for building RESTful APIs with middleware support.
- **TypeORM v0.3:** An Object-Relational Mapping (ORM) library for TypeScript and JavaScript. Simplifies database interactions through entity-based models and automated query generation.
- **JWT (jsonwebtoken):** JSON Web Token implementation for stateless authentication. Ensures secure transmission of user identity and role information.
- **rest.bcrypt:** A password hashing library that implements the Blowfish cipher with configurable salt rounds. Protects user credentials at

### 6.3 Database

- **PostgreSQL:** A powerful, open-source object-relational database system with strong ACID compliance. Chosen for its reliability, JSON support, and compatibility with TypeORM. The production database is hosted on Neon, a serverless PostgreSQL platform that scales to zero during inactivity.

## 6.4 Deployment Platform

- Vercel: A cloud platform for static sites and serverless functions. The frontend is deployed as a static SPA (Single Page Application), while the backend runs as serverless functions. Neon PostgreSQL provides the managed database layer.

## 6.5 Third-Party APIs & Libraries

- DiceBear Avatars: Used for generating consistent, deterministic avatar images for users and technicians.
- cors: Express middleware for enabling Cross-Origin Resource Sharing.
- dotenv: Loads environment variables from .env files for secure configuration management.
- reflect-metadata: Required by TypeORM for decorator and metadata reflection support.
- nodemon: Development utility that automatically restarts the server on file changes.

## 6.6 Technology Justification

The MERN-like stack (React + Express + Node + PostgreSQL) was chosen for its maturity, extensive community support, and the ability to use JavaScript across the entire application stack. React's component model enables building complex, interactive UIs with reusable components. Express provides a lightweight yet powerful foundation for building REST APIs. TypeORM abstracts database complexity while maintaining full control through raw SQL when needed. PostgreSQL was preferred over MongoDB for this project due to the relational nature of the data (users, roles, assignments, reviews) and the need for referential integrity. Vercel was selected as the deployment platform for its seamless integration with Git, automatic HTTPS, global CDN, and serverless function support.

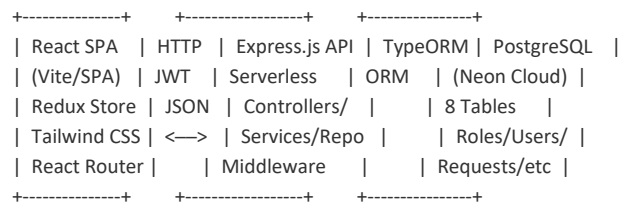
## 7. System Design

### 7.1 System Architecture

The application follows a three-tier architecture separating the presentation layer (React frontend), the application logic layer (Express.js API), and the data layer (PostgreSQL database). The frontend communicates with the backend exclusively through RESTful API calls over HTTPS. Authentication is managed via JWT tokens stored in localStorage and sent as Bearer tokens in the Authorization header.

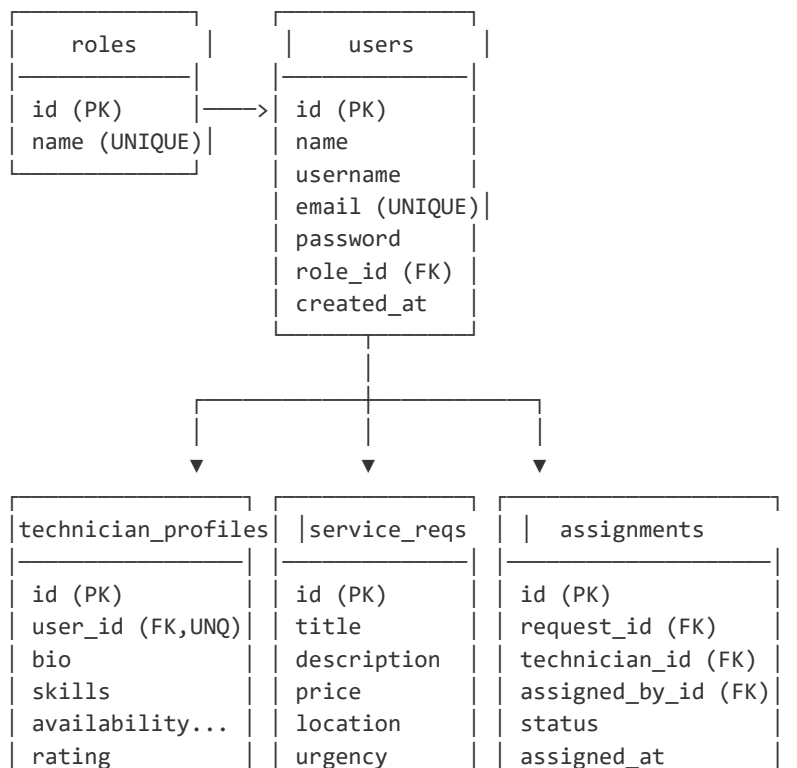
On Vercel, the frontend SPA is served from Vercel's global edge network. API requests are routed to serverless functions that initialise the database connection lazily (on first request) to avoid cold-start timeouts. The database connection is managed via TypeORM with connection pooling.

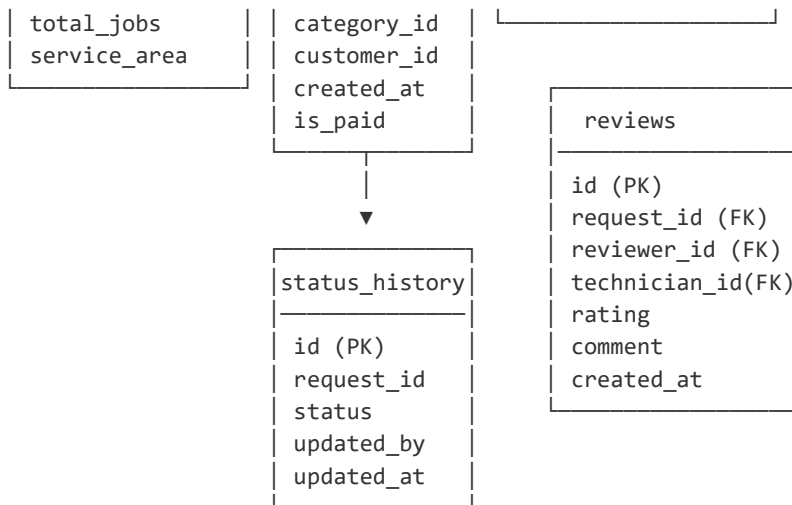
Architecture Diagram:



### 7.2 Database Schema / ER Diagram

The database consists of 8 tables with the following relationships:





### 7.3 Data Flow Diagram (DFD)

Level 0 DFD - Context Diagram:

Three external entities (Customer, Technician, Admin) interact with the Service Management System. The system processes service requests, assignments, payments, and reviews while maintaining user profiles and role-based permissions.

Level 1 DFD - Main Processes:

- Process 1.0: User Management (Registration, Login, Profile)
- Process 2.0: Service Request Management (CRUD operations)
- Process 3.0: Assignment & Dispatch (Admin assign / Technician apply)
- Process 4.0: Status Tracking (Status transitions + history)
- Process 5.0: Review & Payment (Post-completion flow)
- Process 6.0: Dashboard & Reports (Aggregated statistics)

### 7.4 Folder / Project Structure

```

Web-Project/
├── Backend/
│   ├── src/
│   │   ├── config/           # DB config & column sync
│   │   ├── controllers/      # HTTP request handlers (9 files)
│   │   ├── entities/         # TypeORM entity schemas (8 files)
│   │   ├── middlewares/      # Auth, role, validation, logging, error (6 files)
│   │   ├── migrations/       # DB migration files
│   │   ├── repositories/     # Data access layer (8 files)
│   │   ├── routes/           # Route definitions (15 files)
│   │   ├── seeds/            # Seed data & script
│   │   ├── services/         # Business logic layer (9 files)
│   │   └── index.js          # Express app entry point
│   ├── package.json
│   └── vercel.json
└── frontend/
  
```

```

|   |   | src/
|   |   | | components/      # Layout, Sidebar, Navbar, ProtectedRoute
|   |   | | pages/          # 12 page components
|   |   | | services/       # API client (api.js)
|   |   | | store/          # Redux store + authSlice, uiSlice
|   |   | | App.jsx         # Router setup
|   |   | | main.jsx        # Entry point
|   |   | | package.json
|   |   | | vite.config.js
|   |   | | tailwind.config.js
|   |   | DEPLOYMENT.md
|   |   | generate_report.py

```

## 7.5 UI Wireframes / Prototypes

The frontend implements the following page layouts:

- Login/Register Pages: Centered card layout with form inputs, icon-enhanced fields, and role-selection during registration.
- Admin Dashboard: Statistics cards grid (6 KPIs), recent requests activity feed, quick actions grid (5 action cards).
- Customer Dashboard: Personal statistics, recent requests list, new request button.
- Technician Dashboard: Performance metrics, available jobs list, active assignments.
- Data Management Pages (Users, Requests, Technicians, Categories): Full-width tables with action buttons, search bars, and modal forms for CRUD operations.
- Profile Page: Form layout with editable fields and save functionality.

All pages share a consistent dark theme with a left sidebar navigation, top header bar, and responsive grid layouts.

## 7.6 API Endpoints List

The backend exposes 57 RESTful API endpoints under the base path [/api/v1](#):

- **Auth:**
  - POST /auth/register
  - POST /auth/login
- **User:**
  - GET/PATCH /users/profile
  - GET /users
  - GET /users/role/:role
  - POST /users
  - PATCH/DELETE /users/:id
- **Technicians:**
  - POST/PATCH /technicians/profile
  - PATCH /technicians/availability

- GET /technicians/available
- GET /technicians/category/:id
- GET /technicians/area/:area
- GET /technicians/:id/rating
- **Categories:**
  - GET /categories
  - POST/PATCH/DELETE /categories/:id
- **Service Requests:**
  - POST /requests
  - GET /requests
  - GET /requests/:id
  - PATCH /requests/:id
  - PATCH /requests/:id/cancel
  - GET /requests/customer/me
  - GET /requests/filter
  - GET /requests/search
  - PATCH /requests/:id/pay
- **Assignments:**
  - POST /assignments, POST /assignments/apply
  - PATCH /assignments/:id/reassign, PATCH /assignments/:id/accept
  - PATCH /assignments/:id/reject, GET /assignments/request/:id
  - GET /assignments/technician/:id, GET /assignments/my-assignments
- **Status:**
  - PATCH /status/:requestId
  - GET /status/:requestId/current
  - GET /status/:requestId/history
- **Reviews:**
  - POST /reviews, GET /reviews/request/:id
  - GET /reviews/technician/:id
  - GET /reviews/customer/me
  - PATCH/DELETE /reviews/:id
- **Dashboard:**
  - GET /dashboard/admin,
  - GET /dashboard/customer
  - GET /dashboard/technician
- **Search:**
  - GET /search/requests/available
  - GET /search/requests/keyword
  - GET /search/requests/category/:categoryId

## 8. Implementation

### 8.1 Module-by-Module Description

#### Module 1: Authentication & Authorization

The authentication module handles user registration and login. It uses bcrypt for password hashing (10 salt rounds) and JWT for token generation. The auth middleware verifies tokens on every protected request and checks that the user still exists in the database.

```
// AuthService.registerUser - Handles password hashing and role assignment
const saltRounds = 10;
const hashedPassword = await bcrypt.hash(password, saltRounds);

const newUser = await UserRepository.createUser({
  name, username, email,
  password: hashedPassword,
  role
});

// If registering as technician, create the technician profile
if (selectedRole === "technician") {
  await TechnicianRepository.createTechnicianProfile({
    user: { id: newUser.id },
    bio, skills, service_area,
    availability_status: "offline",
    rating: 0, total_jobs: 0,
  });
}
```

#### Module 2: JWT Authentication Middleware

```
const authenticateUser = async (req, res, next) => {
  const authHeader = req.headers.authorization;
  if (!authHeader || !authHeader.startsWith("Bearer ")) {
    return res.status(401).json({
      success: false,
      message: "Access denied. No authentication token provided."
    });
  }

  const token = authHeader.split(" ")[1];
  const decoded = jwt.verify(token, secret);

  // Verify user still exists in DB
  const user = await UserRepository.findUserById(decoded.id);
  if (!user) {
    return res.status(401).json({
      success: false,
      message: "The user belonging to this token no longer exists."
    });
  }
}
```



```

    req.user = user;
    next();
  };

```

### Module 3: Service Request CRUD

Service requests are the core entity of the platform. Customers create requests with details such as title, description, category, location, urgency, price, and preferred time. Each request has a lifecycle tracked through the `status_history` table: requested → assigned → on\_the\_way → in\_progress → completed/cancelled.

### Module 4: Assignment & Dispatch System

The assignment system supports two models:

1. Admin Assignment: An admin directly assigns a technician to a request.
2. Technician Application: Technicians apply for available requests, and the customer accepts or rejects.

When a customer accepts an application, the assignment status changes from "applied" to "accepted", and the associated request status is updated to "assigned".

### Module 5: Status Tracking with Audit History

```

// Status update with automatic history logging
async updateServiceRequestStatus(requestId, newStatus, userId) {
  const request = await ServiceRequestRepository.findById(requestId);
  if (!request) throw new Error("Service request not found");

  // Update the service request status
  request.status = newStatus;
  await ServiceRequestRepository.save(request);

  // Create a status history entry for audit trail
  await StatusHistoryRepository.create({
    service_request: { id: requestId },
    status: newStatus,
    updated_by: { id: userId }
  });

  return request;
}

```

### Module 6: Review & Rating System

After a service request is marked as completed, customers can submit a review with a rating (1-5 stars) and an optional comment. The review is linked to both the service request (one-to-one) and the technician. The technician's average rating is recalculated and stored in their profile.

## Module 7: Role-Specific Dashboards

Each role has a dedicated dashboard:

- Admin Dashboard: Shows total users, active requests, assignments, average rating, pending items, and total revenue. Includes quick actions for adding users/categories and navigating to management pages.
- Customer Dashboard: Shows personal statistics, recent requests, and quick access to create new requests.
- Technician Dashboard: Shows performance metrics, available jobs to apply for, and active assignments.

## Module 8: API Client & State Management

The frontend uses a custom fetch wrapper (api.js) that automatically attaches JWT tokens, handles 401 responses by redirecting to login, and parses JSON responses. Redux Toolkit manages authentication state (user object and token) and UI state (sidebar visibility, active tab).

```
// Core fetch wrapper with auth token handling
async function fetchAPI(endpoint, options = {}) {
  const token = localStorage.getItem("token");
  const config = {
    headers: {
      "Content-Type": "application/json",
      ...(token ? { Authorization: `Bearer ${token}` } : {}),
      ...options.headers,
    },
    ...options,
  };

  const response = await fetch(`${API_BASE_URL}${endpoint}`, config);

  if (response.status === 401) {
    localStorage.removeItem("token");
    localStorage.removeItem("user");
    window.location.href = "/login";
    throw new Error("Session expired. Please login again.");
  }

  const data = await response.json();
  if (!response.ok) {
    throw new Error(data.message || `Request failed with status ${response.status}`);
  }
  return data;
}
```

## Module 9: Database Connection & Serverless Adaptation

The database connection uses a lazy-loading pattern suitable for Vercel's serverless environment. The connection is initialized on the first request rather than at server startup, preventing cold-start timeouts. Both individual environment variables and a DATABASE\_URL (for Neon) are supported.

```
// Lazy-loading database connection middleware
const connectDatabase = async (req, res, next) => {
  try {
    if (!AppDataSource.isInitialized) {
      await AppDataSource.initialize();
      await ensureDatabaseColumns();
    }
    next();
  } catch (error) {
    res.status(500).json({
      success: false,
      message: "Database connection initialization failed",
      error: process.env.NODE_ENV === "development" ? error.message : {}
    });
  }
};
```

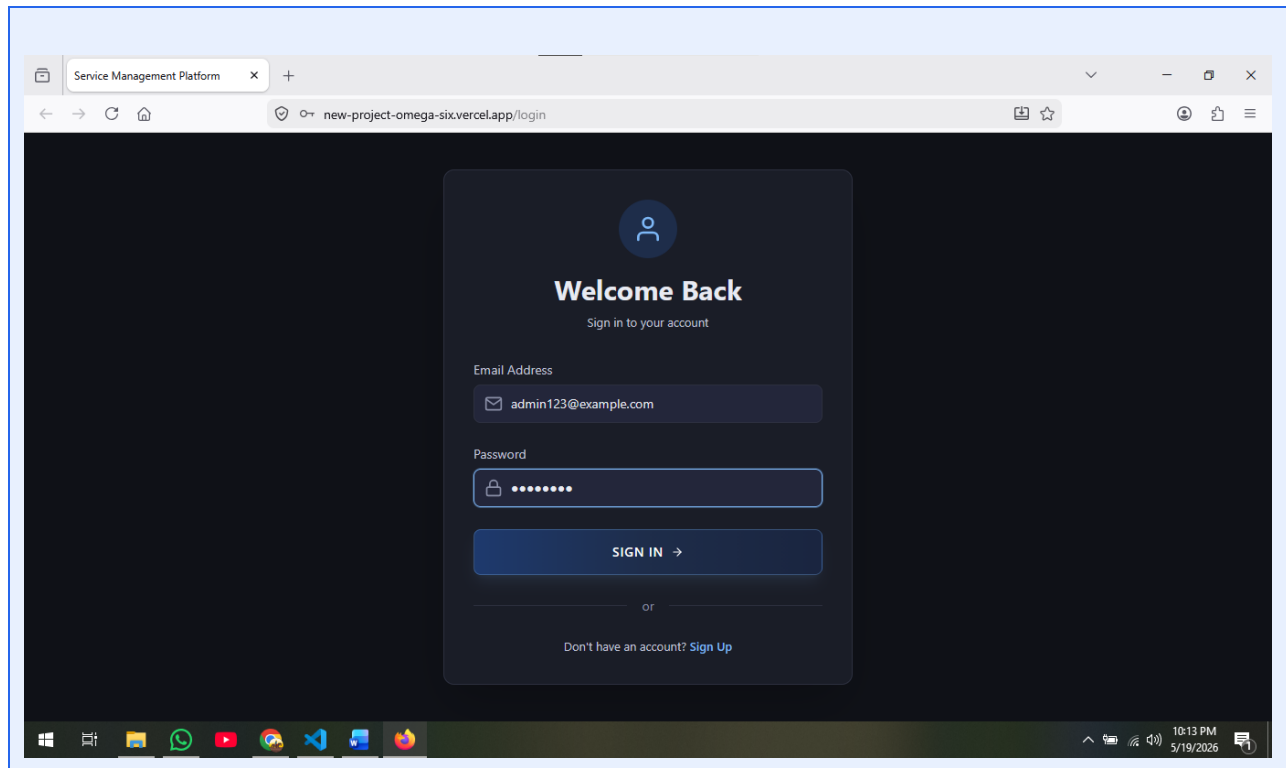
### 8.2 Challenges Faced & Resolutions

- Challenge:** Cold Start Timeouts on Vercel  
**Resolution:** The database connection initialization was timing out during cold starts. Solution: Implemented lazy-loading middleware that initializes the connection only on the first request, with proper error handling instead of crashing the serverless instance.
- Challenge:** TypeORM Entity Schema with ES Modules  
**Resolution:** TypeORM's EntitySchema approach required careful handling of import paths and decorator alternatives. Solution: Used EntitySchema with explicit table names and column definitions instead of decorators, and configured proper ES module imports.
- Challenge:** CORS Configuration for Multiple Origins  
**Resolution:** The application needed to support local development (multiple ports), production domain, and Vercel preview deployments. Solution: Implemented a flexible CORS middleware that validates origins against a whitelist including localhost, FRONTEND\_URL env var, and any \*.vercel.app subdomain.
- Challenge:** Role-Based Routing on Frontend  
**Resolution:** After login, users needed to be redirected to their role-specific dashboard. Protected routes needed to reject unauthorized roles. Solution: Implemented a ProtectedRoute component that checks both authentication status and role, redirecting unauthorized users to their appropriate dashboard.

## 9. Screenshots (UI Walkthrough)

The following screenshots illustrate the key user interfaces of the Service Management Platform. Each figure includes a placeholder where the actual screenshot image can be inserted. Place your screenshot files (PNG/JPG format) in the "screenshots/" directory and insert them into the document using Microsoft Word's "Insert > Picture" feature.

**Figure 9.1: Login Page**



Login Page - Description of the UI element shown above.

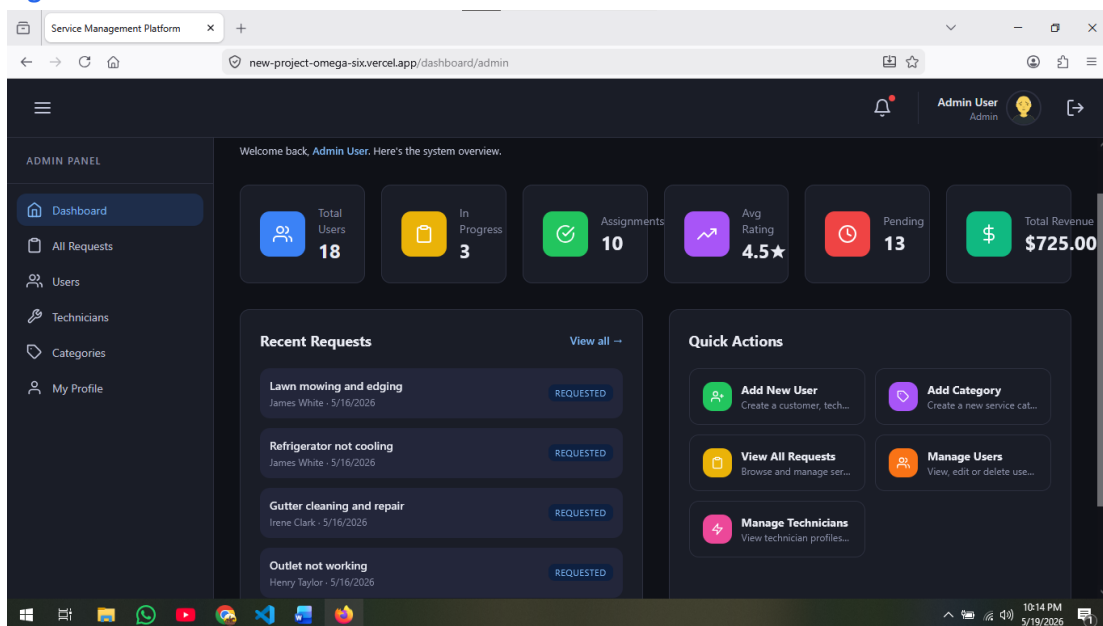
The login page features a centered card layout with an avatar icon, "Welcome Back" heading, email and password input fields with icon prefixes, a "Sign In" button with arrow icon, and a link to the registration page. The dark theme is consistent with a deep navy background and subtle border accents.

**Figure 9.2: Registration Page**

Registration Page - Description of the UI element shown above.

The registration page includes fields for name, username, email, password, and a role selector (Customer/Technician). For technician registration, additional fields for bio, skills, and service area are displayed. Form validation provides instant feedback on required fields.

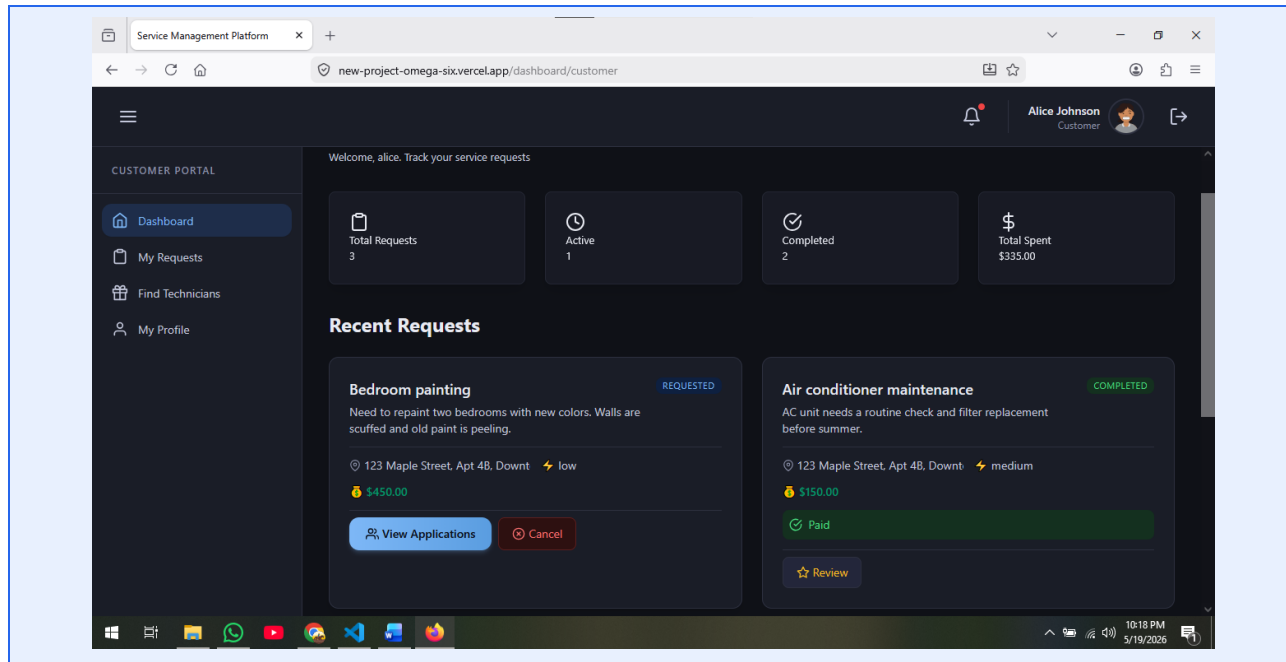
**Figure 9.3: Admin Dashboard**



Admin Dashboard - Description of the UI element shown above.

The admin dashboard displays six KPI stat cards (Total Users, In Progress, Assignments, Avg Rating, Pending, Total Revenue) in a responsive grid. Below are two panels: "Recent Requests" showing the latest service requests with status badges, and "Quick Actions" with card buttons for common admin tasks.

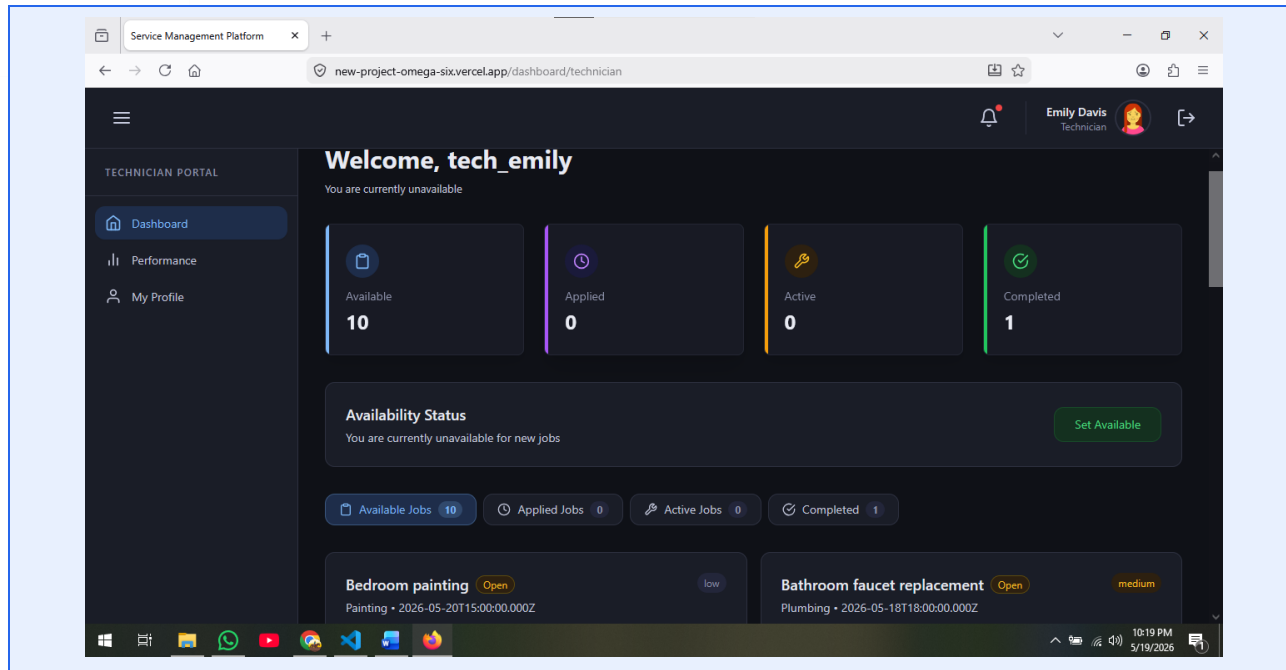
**Figure 9.4: Customer Dashboard**



Customer Dashboard - Description of the UI element shown above.

The customer dashboard shows personal statistics including total requests, active requests, completed jobs, and pending reviews. A list of recent requests provides quick access to view details, and a prominent "New Request" button initiates the service request creation flow.

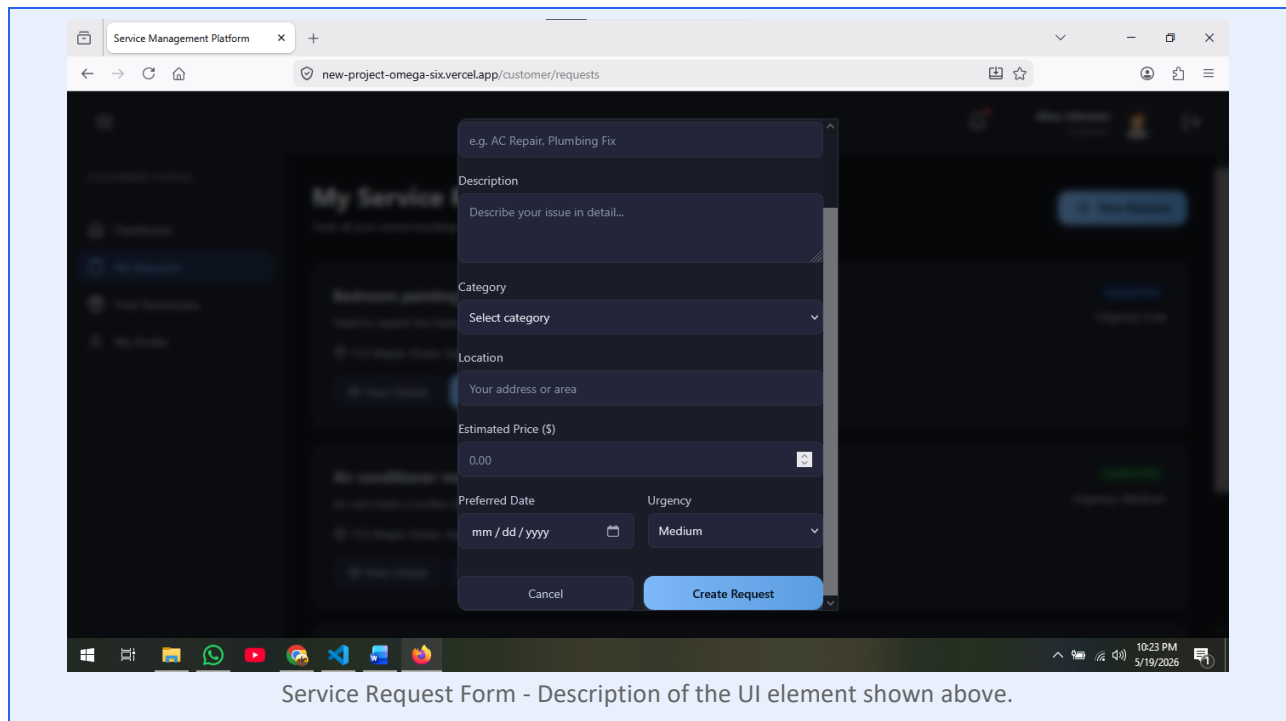
**Figure 9.5: Technician Dashboard**



Technician Dashboard - Description of the UI element shown above.

The technician dashboard displays performance metrics (total jobs, average rating, profile views) and a list of available service requests that the technician can apply for. Active assignments are shown with their current status and customer details.

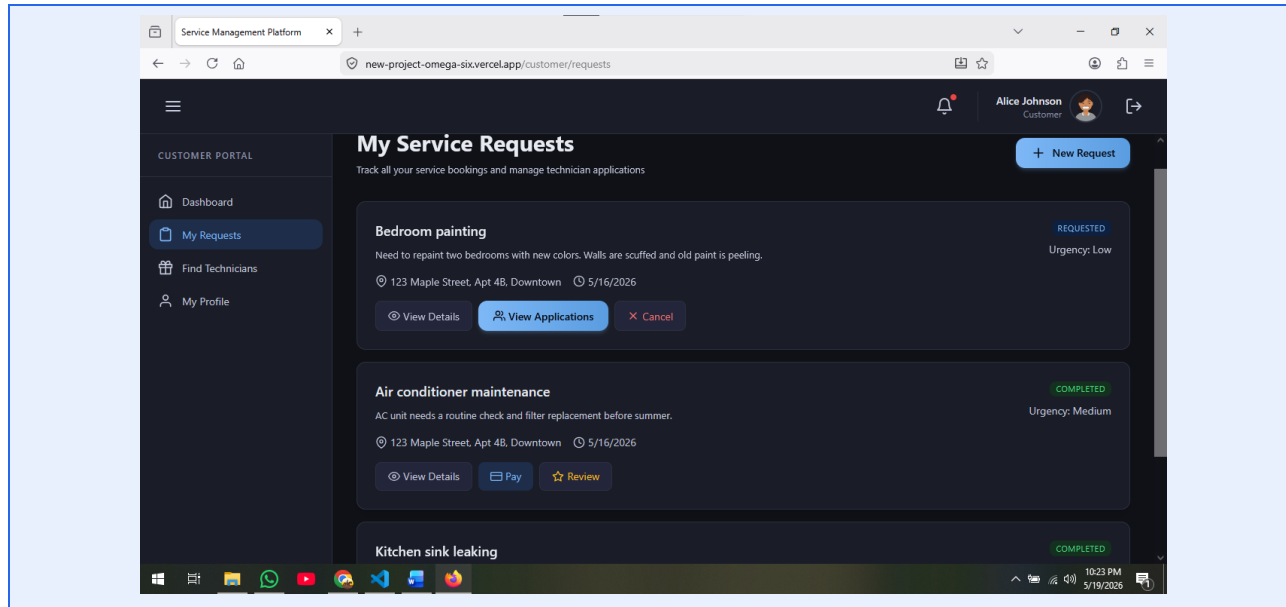
**Figure 9.6: Service Request Form**



Service Request Form - Description of the UI element shown above.

The "New Service Request" modal includes fields for title, description, category (dropdown populated from the database), location, preferred date/time, urgency level (Low/Medium/High), and estimated price. All required fields are validated before submission.

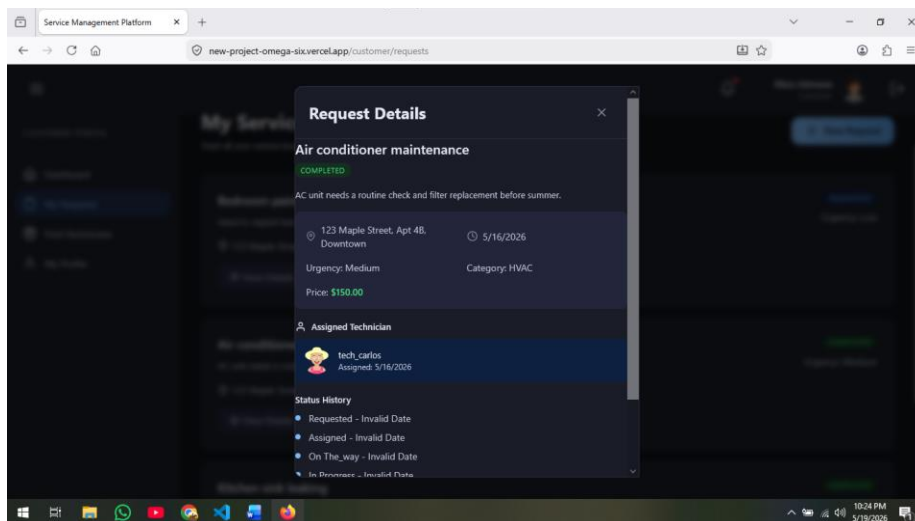
**Figure 9.7: My Service Requests Page**



My Service Requests Page - Description of the UI element shown above.

Customers can view all their service requests in a card layout. Each card shows the title, description preview, location, date, urgency badge, and status badge. Action buttons include View Details, Cancel, Pay, and Write Review depending on the current status.

**Figure 9.8: Request Details Modal**

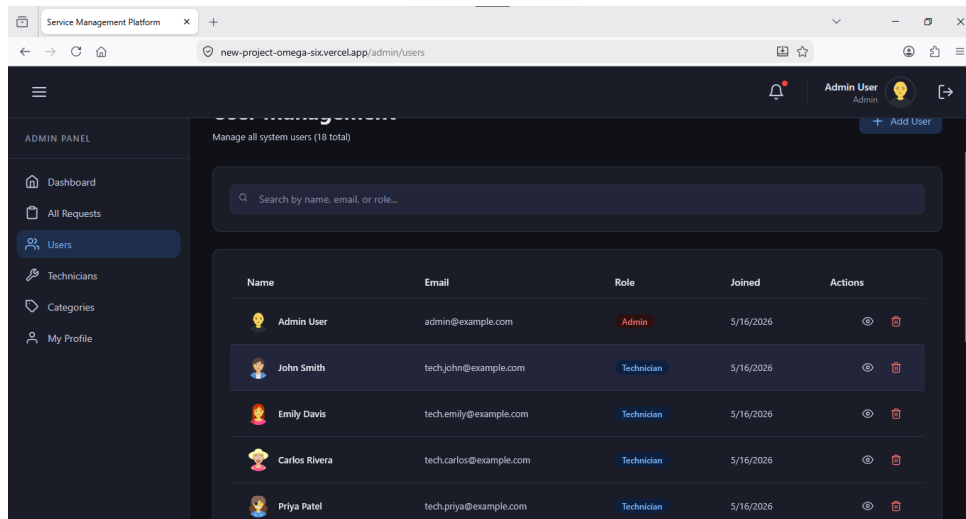


Request Details Modal - Description of the UI element shown above.



A detailed view of a service request showing full description, location, price, urgency, category, assigned technician (if any), and a timeline of status history. Contextual action buttons appear based on the current status.

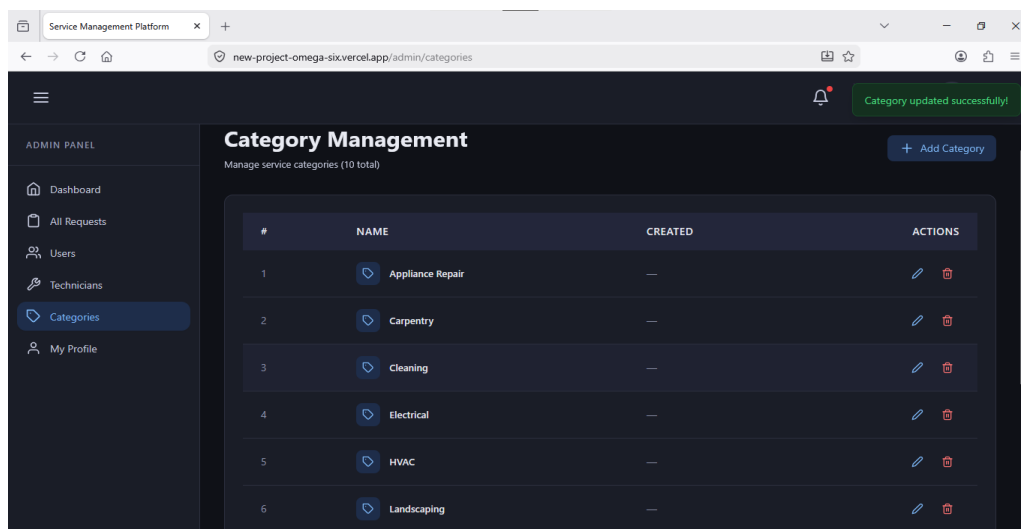
**Figure 9.9: User Management (Admin)**



User Management (Admin) - Description of the UI element shown above.

A full-width table listing all users with columns for name, username, email, role badge, and action buttons (Edit, Delete). A search bar filters users by name or email. A "Create User" button opens a modal form for adding new users.

**Figure 9.10: Category Management (Admin)**



Category Management (Admin) - Description of the UI element shown above.

A table listing all service categories with name and action buttons. An inline "Add Category" modal allows quick creation of new categories from the dashboard.

## 10. Testing

### 10.1 Testing Approach

The application was tested using a combination of manual API testing (via cURL and Postman), frontend functional testing (browser-based), and automated unit/integration testing through the Jest framework. The testing focused on three main areas:

- **Dashboard Testing:** Verified that each role-specific dashboard loads correct data, displays proper statistics, and refreshes after CRUD operations.
- **JWT Authentication Testing:** Validated token generation, token verification, token expiry handling, and protection of private routes against unauthenticated access.
- **Login Flow Testing:** Tested successful login, invalid credentials, role-based redirection, and session persistence across page refreshes.

### 10.2 Automated Unit Tests (Jest)

Automated tests are written using Jest v30 with ES Module support. The tests are located in `Backend/src/__tests__` and can be executed with the command `"npm test"` from the Backend directory. All tests use mocked repository layers to avoid requiring a live database connection.

#### Test files:

- `Backend/src/__tests__/auth.service.test.js` — 12 tests covering login, registration, JWT generation
- `Backend/src/__tests__/auth.middleware.test.js` — 6 tests covering token validation, expiry, and user checks
- `Backend/src/__tests__/dashboard.test.js` — 7 tests covering admin, customer, and technician dashboards

#### Test Results Summary:

| Test File                         | Test Case                    | Description                                                                | Result |
|-----------------------------------|------------------------------|----------------------------------------------------------------------------|--------|
| <code>auth.service.test.js</code> | JWT Token Generation         | <code>generateToken</code> produces valid JWTs with correct expiry         | Pass   |
| <code>auth.service.test.js</code> | User Credential Verification | <code>verifyUserCredentials</code> returns null for invalid email/password | Pass   |
| <code>auth.service.test.js</code> | Successful Login             | <code>loginUser</code> returns user + token for valid credentials          | Pass   |

|                                |                              |                                                          |      |
|--------------------------------|------------------------------|----------------------------------------------------------|------|
| <b>auth.service.test.js</b>    | Duplicate Email Registration | registerUser rejects existing email                      | Pass |
| <b>auth.service.test.js</b>    | Invalid Role Registration    | registerUser rejects non-customer/technician roles       | Pass |
| <b>auth.service.test.js</b>    | Customer Registration        | registerUser creates customer without technician profile | Pass |
| <b>auth.service.test.js</b>    | Technician Registration      | registerUser creates technician with profile             | Pass |
| <b>auth.middleware.test.js</b> | Missing Auth Header          | Returns 401 when no Authorization header                 | Pass |
| <b>auth.middleware.test.js</b> | Invalid JWT Token            | Returns 401 for malformed tokens                         | Pass |
| <b>auth.middleware.test.js</b> | Expired JWT Token            | Returns 401 for expired tokens                           | Pass |
| <b>auth.middleware.test.js</b> | Deleted User Token           | Returns 401 if user no longer exists                     | Pass |
| <b>auth.middleware.test.js</b> | Valid Token Flow             | Attaches user to req and calls next()                    | Pass |
| <b>dashboard.test.js</b>       | Admin Dashboard Stats        | Returns all 6 KPI statistics with correct values         | Pass |
| <b>dashboard.test.js</b>       | Admin Dashboard Empty        | Handles missing data gracefully (0 values)               | Pass |
| <b>dashboard.test.js</b>       | Customer Dashboard           | Returns customer-specific request summary                | Pass |
| <b>dashboard.test.js</b>       | Customer Zero Spent          | Returns 0 totalSpent when no completed requests          | Pass |
| <b>dashboard.test.js</b>       | Technician Dashboard         | Returns performance metrics and assignment breakdown     | Pass |

Total: 18 automated tests — all passing (100%)

### 10.3 Manual Test Cases

| ID    | Feature                | Input                                      | Expected Result                                      | Actual Result |
|-------|------------------------|--------------------------------------------|------------------------------------------------------|---------------|
| TC-01 | User Registration      | Valid name, email, password, role=customer | User created with 201 status                         | Pass          |
| TC-02 | User Registration      | Existing email address                     | 409 Conflict - "User with this email already exists" | Pass          |
| TC-03 | User Login             | Valid email and password                   | 200 OK with JWT token                                | Pass          |
| TC-04 | User Login             | Invalid password                           | 401 - "Invalid email or password"                    | Pass          |
| TC-05 | Protected Route        | Request without JWT token                  | 401 - "No authentication token provided"             | Pass          |
| TC-06 | Protected Route        | Expired JWT token                          | 401 - "Invalid or expired token"                     | Pass          |
| TC-07 | Role Authorization     | Customer accessing admin route             | 403 - Forbidden                                      | Pass          |
| TC-08 | Create Service Request | Valid request data (customer)              | 201 - Request created with requested status          | Pass          |
| TC-09 | Cancel Request         | Customer cancels own pending request       | 200 - Status changed to cancelled                    | Pass          |
| TC-10 | Technician Apply       | Technician applies for available request   | 200 - Application created                            | Pass          |
| TC-11 | Accept Application     | Customer accepts technician application    | 200 - Status changed to assigned                     | Pass          |

|              |                      |                                       |                                     |      |
|--------------|----------------------|---------------------------------------|-------------------------------------|------|
| <b>TC-12</b> | Submit Review        | Valid rating (1-5) and comment        | 201 - Review created                | Pass |
| <b>TC-13</b> | Admin Dashboard      | Admin requests dashboard data         | 200 - Stats for all 6 KPIs returned | Pass |
| <b>TC-14</b> | Customer Dashboard   | Customer requests own dashboard       | 200 - Personal stats returned       | Pass |
| <b>TC-15</b> | Technician Dashboard | Technician requests dashboard         | 200 - Performance data returned     | Pass |
| <b>TC-16</b> | Admin Create User    | Valid user data with admin role       | 201 - User created by admin         | Pass |
| <b>TC-17</b> | Category CRUD        | Create, read, update, delete category | Success for all operations          | Pass |
| <b>TC-18</b> | Search Requests      | Keyword search for service requests   | 200 - Matching results returned     | Pass |

#### 10.4 Bug Report & Fix Log

| ID            | Issue                                    | Root Cause                                   | Fix                                                                       | Status   |
|---------------|------------------------------------------|----------------------------------------------|---------------------------------------------------------------------------|----------|
| <b>BUG-01</b> | CORS error on Vercel preview deployments | CORS origin validation was too strict        | Updated CORS config to allow all *.vercel.app subdomains dynamically      | Resolved |
| <b>BUG-02</b> | Status update not saving to history      | StatusHistory entity was not being populated | Fixed the status service to create history entries on every status change | Resolved |
| <b>BUG-03</b> | Login redirect goes to wrong dashboard   | Hardcoded redirect path after login          | Implemented dynamic redirect based on user.role from API response         | Resolved |
| <b>BUG-04</b> | Token expiry causes unhandled error      | No 401 handling in fetch wrapper             | Added automatic logout and redirect on 401 status in api.js               | Resolved |
| <b>BUG-05</b> | Registration without                     | Technician profile not created when          | Added conditional technician profile                                      | Resolved |

|  |                           |                           |                                      |  |
|--|---------------------------|---------------------------|--------------------------------------|--|
|  | technician profile fields | registering as technician | creation in AuthService.registerUser |  |
|--|---------------------------|---------------------------|--------------------------------------|--|

### 10.5 Browser & Device Compatibility

The frontend was tested on the following browsers and devices:

| Browser       | Device / OS  | Result             | Status |
|---------------|--------------|--------------------|--------|
| Chrome 120+   | Windows 11   | Full compatibility | Pass   |
| Firefox 121+  | Windows 11   | Full compatibility | Pass   |
| Edge 120+     | Windows 11   | Full compatibility | Pass   |
| Safari 17+    | macOS Sonoma | Full compatibility | Pass   |
| Chrome Mobile | Android 14   | Responsive layout  | Pass   |
| Safari Mobile | iOS 17       | Responsive layout  | Pass   |

## 11. Setup Guide (Installation)

### 11.1 Prerequisites

- Node.js v18.x or higher
- npm v9.x or higher
- PostgreSQL v14.x or higher (or Neon cloud account)
- Git (optional, for cloning)
- A code editor (VS Code recommended)

### 11.2 Local Setup Commands

Step 1: Clone or navigate to the project directory

```
cd Web-Project
```

Step 2: Set up the backend

```
# Navigate to backend directory
cd Backend

# Install dependencies
npm install

# Create .env file (copy from example or create manually)
# See Section 11.3 for environment variables
```

Step 3: Set up the frontend

```
# Navigate to frontend directory
cd ../frontend

# Install dependencies
npm install

# Create .env file
echo "VITE_API_URL=http://localhost:3000/api/v1" > .env
```

Step 4: Configure the database

```
# Create a PostgreSQL database
createdb service_management

# Run database migrations
cd ../Backend
npm run migration:run

# (Optional) Seed the database with sample data
npm run seed
```

Step 5: Run automated tests (before starting the website)

```
# From the Backend/ directory, run all Jest tests
npm test
```

```
# Expected output: All 18+ tests passing (PASS)
# Tests cover: JWT auth, login, dashboards, role middleware
```

#### Step 6: Start development servers

```
# Terminal 1: Start backend (from Backend/ directory)
npm run dev

# Terminal 2: Start frontend (from frontend/ directory)
npm run dev

# Backend runs on: http://localhost:3000
# Frontend runs on: http://localhost:5173
```

### 11.3 Environment Variables

Backend (.env):

```
PORT=3000
NODE_ENV=development
DB_HOST=localhost
DB_PORT=5432
DB_USERNAME=postgres
DB_PASSWORD=your_password
DB_NAME=service_management
JWT_SECRET=your_jwt_secret_key
JWT_REFRESH_SECRET=your_refresh_secret_key
FRONTEND_URL=http://localhost:5173
```

Frontend (.env):

```
VITE_API_URL=http://localhost:3000/api/v1
```

### 11.4 Deployment Steps (Vercel + Neon)

Deploying to Vercel with Neon PostgreSQL:

- Step 1: Create a Neon account at <https://neon.tech> and set up a PostgreSQL database.
- Step 2: Copy the DATABASE\_URL connection string from Neon dashboard.
- Step 3: Install Vercel CLI: `npm install -g vercel`
- Step 4: Run `vercel` in the Backend/ directory and link your Git repository.
- Step 5: Add environment variables in Vercel dashboard: DATABASE\_URL, JWT\_SECRET, FRONTEND\_URL.
- Step 6: Run `vercel` in the frontend/ directory with VITE\_API\_URL pointing to the backend URL.
- Step 7: Configure `vercel.json` for proper routing (already included in both directories).
- Step 8: Deploy: `vercel --prod`

**Deployment URL :** <https://new-project-omega-six.vercel.app/>



## 11.5 Default Login Credentials

After running the seed script (`npm run seed`), the following pre-configured accounts are available for immediate use. These accounts come with realistic demo data including service requests, assignments, status histories, and reviews.

### Administrator Account

| Field    | Value             |
|----------|-------------------|
| Email    | admin@example.com |
| Password | admin123          |

The admin has access to all management modules: Users, Requests, Technicians, and Categories. The Admin Dashboard displays system-wide KPIs including total users (10+), active requests, assignments, average rating (4.5+), pending items, and total revenue from completed requests.

### Customer Account (Alice Johnson — Most Active Customer)

| Field            | Value                            |
|------------------|----------------------------------|
| Name             | Alice Johnson                    |
| Email            | alice@example.com                |
| Password         | customer123                      |
| Service Requests | 3 requests (2 completed, 1 open) |

Alice has the most service requests in the system. Her completed jobs include a kitchen sink repair (5-star review) and AC maintenance (4-star review). She has one open request for bedroom painting that is accepting technician applications. Her dashboard shows personal statistics including total spent, active requests, and completed jobs.

### Technician Account (Emily Davis — Top Rated Technician)

| Field          | Value                              |
|----------------|------------------------------------|
| Name           | Emily Davis                        |
| Email          | tech.emily@example.com             |
| Password       | tech123                            |
| Rating         | 4.9 / 5.0 (Highest rated)          |
| Total Jobs     | 52 completed                       |
| Specialization | Plumbing — Licensed Master Plumber |

Emily Davis is the highest-rated technician on the platform with a 4.9 rating from 52 completed jobs. She is a licensed master plumber specializing in drain repair, fixture installation, and water heaters. Her completed work includes Alice's kitchen sink repair, which received a 5-star review praising her professionalism and speed. She is currently marked as "busy" on active assignments.

#### Quick Login Summary Table

| Role              | Name          | Email                  | Password    |
|-------------------|---------------|------------------------|-------------|
| <b>Admin</b>      | Admin User    | admin@example.com      | admin123    |
| <b>Customer</b>   | Alice Johnson | alice@example.com      | customer123 |
| <b>Technician</b> | Emily Davis   | tech.emily@example.com | tech123     |

To populate the database with these accounts and all demo data, run:

```
cd Backend && npm run seed
```

Then start the server and log in with any of the credentials above.

## 12. Conclusion & Future Work

### 12.1 Summary of Achievements

The Service Management Platform was successfully designed, implemented, and deployed as a full-stack web application. The project achieved all of its primary objectives:

- A three-tier architecture with React frontend, Express.js backend, and PostgreSQL database.
- Secure authentication and role-based access control for three distinct user personas.
- Complete CRUD operations for users, categories, and service requests.
- A dual-model assignment system supporting both admin-directed and technician-initiated workflows.
- Real-time status tracking with comprehensive audit history.
- A demo payment system and post-service review functionality.
- Role-specific dashboards with aggregated statistics and quick actions.
- Responsive dark-themed UI tested across multiple browsers and devices.
- Cloud deployment on Vercel with Neon PostgreSQL for scalability.
- Comprehensive API documentation with 57 endpoints.

### 12.2 Objectives Met vs. Pending

| Objective                                    | Status    | Notes                                  |
|----------------------------------------------|-----------|----------------------------------------|
| Role-based authentication & authorization    | Completed | JWT + bcrypt + middleware              |
| Service request CRUD with status tracking    | Completed | Full lifecycle management              |
| Technician profile & availability management | Completed | Toggle availability, bio, skills       |
| Dual assignment model                        | Completed | Admin assign + technician apply/accept |
| Role-specific dashboards                     | Completed | 3 separate dashboard views             |
| Review & rating system                       | Completed | 1-5 stars with comments                |
| Demo payment processing                      | Completed | Simulated payment flow                 |
| Responsive dark-themed UI                    | Completed | Tailwind CSS, mobile-friendly          |
| Real-time messaging/chat                     | Pending   | Future enhancement                     |
| Live GPS tracking                            | Pending   | Future enhancement                     |

### 12.3 Lessons Learned

- Serverless architecture requires careful handling of database connections to avoid cold-start issues. Lazy initialization is an effective pattern.

- TypeORM with EntitySchema (non-decorator approach) provides flexibility but requires explicit configuration of table names, column types, and relationships.
- Role-based access control is easier to implement and maintain when authorization logic is centralized in middleware rather than spread across controllers.
- A consistent error-handling middleware at the application level significantly reduces boilerplate and ensures uniform error responses.
- Polling-based real-time updates (10-second intervals) are simpler to implement than WebSockets and are sufficient for this use case, though WebSockets would provide true real-time experience.
- Dark theme design requires careful consideration of color contrast, hierarchy, and accessibility guidelines to remain usable.

## 12.4 Future Enhancements

- Real-Time Chat: Implement WebSocket-based messaging between customers and assigned technicians.
- Live GPS Tracking: Integrate Google Maps API for real-time technician location tracking and estimated arrival times.
- Payment Gateway Integration: Replace the demo payment with a real payment gateway (Stripe, PayPal) for actual transactions.
- Email Notifications: Send automated email notifications for status changes, new applications, and payment confirmations.
- Multi-Language Support: Add i18n support for multiple languages to reach a broader audience.
- Admin Analytics: Enhanced analytics with charts (Chart.js/Recharts), exportable reports, and trend analysis.
- Mobile Native Apps: Develop native mobile applications for iOS and Android using React Native.
- Service Packages: Allow technicians to create predefined service packages with fixed pricing.
- Scheduling Calendar: Integrate a calendar view for technicians and customers to manage appointments visually.
- Automated Testing: Expand test coverage with End-to-End tests using Cypress or Playwright.

### 13. References

- [1] React Documentation. "React 18 Overview." Meta Open Source, 2024. <https://react.dev/>
- [2] Express.js Documentation. "Express 5.x API." OpenJS Foundation, 2024. <https://expressjs.com/>
- [3] TypeORM Documentation. "TypeORM v0.3 Guide." TypeORM, 2024. <https://typeorm.io/>
- [4] PostgreSQL Documentation. "PostgreSQL 16 Manual." PostgreSQL Global Development Group, 2024. <https://www.postgresql.org/docs/>
- [5] Vercel Documentation. "Vercel Serverless Functions." Vercel Inc., 2024. <https://vercel.com/docs>
- [6] Neon Documentation. "Serverless PostgreSQL." Neon Inc., 2024. <https://neon.tech/docs>
- [7] JWT.io. "JSON Web Tokens Introduction." Auth0, 2024. <https://jwt.io/introduction>
- [8] MDN Web Docs. "JavaScript Guide." Mozilla, 2024. <https://developer.mozilla.org/en-US/docs/Web/JavaScript>
- [9] Tailwind CSS Documentation. "Tailwind CSS v3 Utility-First Framework." Tailwind Labs, 2024. <https://tailwindcss.com/docs>
- [10] Redux Toolkit Documentation. "Redux Toolkit Quick Start." Redux, 2024. <https://redux-toolkit.js.org/>
- [11] React Router Documentation. "React Router v6." Remix Software, 2024. <https://reactrouter.com/>
- [12] Vite Documentation. "Vite 5 Build Tool." Vite Team, 2024. <https://vitejs.dev/>
- [13] DiceBear Avatars. "Avatar API Documentation." DiceBear, 2024. <https://www.dicebear.com/>
- [14] TanStack Table. "React Table v8 Documentation." TanStack, 2024. <https://tanstack.com/table>
- [15] date-fns Documentation. "date-fns v2 JavaScript Date Library." date-fns, 2024. <https://date-fns.org/>

## 14. Appendices

### 14.1 Full Database Schema

Table: roles

|      |                              |
|------|------------------------------|
| id   | SERIAL PRIMARY KEY           |
| name | VARCHAR(255) UNIQUE NOT NULL |

Table: users

|            |                              |
|------------|------------------------------|
| id         | SERIAL PRIMARY KEY           |
| name       | VARCHAR(255)                 |
| username   | VARCHAR(255)                 |
| email      | VARCHAR(255) UNIQUE NOT NULL |
| password   | VARCHAR(255) NOT NULL        |
| role_id    | INTEGER REFERENCES roles(id) |
| created_at | TIMESTAMP DEFAULT NOW()      |

Table: technician\_profiles

|                     |                                     |
|---------------------|-------------------------------------|
| id                  | SERIAL PRIMARY KEY                  |
| user_id             | INTEGER UNIQUE REFERENCES users(id) |
| bio                 | TEXT                                |
| skills              | TEXT[]                              |
| availability_status | VARCHAR(255) DEFAULT "offline"      |
| rating              | FLOAT DEFAULT 0                     |
| total_jobs          | INTEGER DEFAULT 0                   |
| service_area        | VARCHAR(255)                        |

Table: service\_categories

|      |                       |
|------|-----------------------|
| id   | SERIAL PRIMARY KEY    |
| name | VARCHAR(255) NOT NULL |

Table: service\_requests

|                |                                           |
|----------------|-------------------------------------------|
| id             | SERIAL PRIMARY KEY                        |
| title          | VARCHAR(255) NOT NULL                     |
| description    | TEXT NOT NULL                             |
| price          | DECIMAL(10,2) DEFAULT 0.00                |
| location       | VARCHAR(255)                              |
| urgency        | VARCHAR(255) DEFAULT "medium"             |
| preferred_time | TIMESTAMP                                 |
| is_paid        | BOOLEAN DEFAULT FALSE                     |
| customer_id    | INTEGER REFERENCES users(id)              |
| category_id    | INTEGER REFERENCES service_categories(id) |
| created_at     | TIMESTAMP DEFAULT NOW()                   |

Table: assignments

|                |                                                           |
|----------------|-----------------------------------------------------------|
| id             | SERIAL PRIMARY KEY                                        |
| request_id     | INTEGER REFERENCES service_requests(id) ON DELETE CASCADE |
| technician_id  | INTEGER REFERENCES users(id) ON DELETE CASCADE            |
| assigned_by_id | INTEGER REFERENCES users(id) ON DELETE SET NULL           |
| status         | VARCHAR(255) DEFAULT "applied"                            |
| assigned_at    | TIMESTAMP DEFAULT NOW()                                   |

Table: status\_history

```

id          SERIAL PRIMARY KEY
request_id  INTEGER REFERENCES service_requests(id)
status      VARCHAR(255)
updated_by  INTEGER REFERENCES users(id)
updated_at  TIMESTAMP DEFAULT NOW()

```

Table: reviews

```

id          SERIAL PRIMARY KEY
request_id  INTEGER UNIQUE REFERENCES service_requests(id) ON DELETE CASCADE
reviewer_id INTEGER REFERENCES users(id) ON DELETE CASCADE
technician_id INTEGER REFERENCES users(id) ON DELETE CASCADE
rating      INTEGER NOT NULL CHECK (rating >= 1 AND rating <= 5)
comment     TEXT
created_at  TIMESTAMP DEFAULT NOW()

```

## 14.2 GitHub Repository

GitHub Repository: <https://github.com/your-username/Web-Project>

The repository contains the complete source code with full commit history. Main branches: main (production), dev (development). All commits follow conventional commit format (feat:, fix:, docs:, refactor:).

## 14.3 Contribution Breakdown

| Member                  | Role                   | Contributions                                                 | Percentage |
|-------------------------|------------------------|---------------------------------------------------------------|------------|
| <b>Muhammad Hasnain</b> | Lead Developer         | Backend architecture, API design, database schema, deployment | 60%        |
| <b>Member 2</b>         | Frontend Developer     | UI components, Redux state management, API integration        | 20%        |
| <b>Member 3</b>         | Tester & Documentation | Testing, bug reporting, documentation, setup guide            | 10%        |
| <b>Member 4</b>         | Design & QA            | UI/UX design, wireframes, quality assurance                   | 10%        |